Q1 Transaction Management

2.5 Points

Consider the following actions taken by transaction T1 on database objects X and Y:

R(X), W(X), R(Y), W(Y)

Q1.1

0.5 Points

1. Which of the following correctly represents the steps of Strict Two-Phase Locking (Strict 2PL)?

   Lock(X), R(X), W(X), Unlock(X), Lock(Y), R(Y), W(Y), Unlock(Y), Commit

   Lock(X), R(X), W(X), Lock(Y), R(Y), W(Y), Unlock(X), Unlock(Y), Commit

   R(X), Lock(X), W(X), R(Y), Lock(Y), Unlock(X), W(Y), Unlock(Y), Commit

   Lock(X), R(X), Lock(Y), Unlock(Y), W(X), R(Y), W(Y), Unlock(X), Commit

Q1.2
0.5 Points

Which schedule violates 2PL (i.e., starts growing again after it begins shrinking)?

X-LOCK(X), X-LOCK(Y), R(X), W(X), R(Y), W(Y), UNLOCK(X), UNLOCK(Y), COMMIT

X-LOCK(X), R(X), W(X), UNLOCK(X), X-LOCK(Y), R(Y), W(Y), UNLOCK(Y), COMMIT

S-LOCK(X), R(X), X-LOCK(X) (upgrade), W(X), S_LOCK(Y), R(Y), X_LOCK(Y)(upgrade), W(Y), UNLOCK(X), UNLOCK(Y), COMMIT

S_LOCK(Y), S-LOCK(X), R(X), X-LOCK(X) (upgrade), X_LOCK(Y) (upgrade), W(X), UNLOCK(X), R(Y), W(Y), UNLOCK(Y), COMMIT

Q1.3
0.5 Points

Given another transaction T2 interleaving with T1, as follows:

	T1	T2
1	R(X)	
2		R(X)
3		W(X)
4	W(X)	
5	R(Y)	
6	W(Y)	
7	Commit	Commit

Please select the statement that is INCORRECT.

There exists a Read-Write conflict in the schedule

There exists a Write-Write conflict in the schedule

This schedule is conflict-serializable

This schedule may not be equivalent to any serial schedule

Q1.4
0.5 Points

Given the below schedule, please select the statement that is INCORRECT.

	T1	T2
1	XLOCK(X)	
2	R(X)	
3	W(X)	
4		XLOCK(X)
8	XLOCK(Y)	
9	R(Y)	
10	W(Y)	
11	UNLOCK(X)	
12	UNLOCK(Y)	
13		R(X)
14		W(X)
15		UNLOCK(X)
16	C	C

This schedule is 2PL

This schedule involves Write-Read conflict, Write-Write conflict, and Read-Write conflict.

This schedule is conflict-serializable

This schedule may not be equivalent to any serial schedule

Q1.5
0.5 Points

4. Strict 2PL is used in many database systems. Which of the following is incorrect about Strict 2PL?

Does not incur cascading aborts

Aborted transactions can be undone by just restoring original values of modified tuples

Can guarantee conflict-serializable schedules

Allows transactions to release locks early to improve concurrency

Easy to implement

Q2

2.5 Points

Suppose we have three transactions T1, T2 and T3 scheduled by the locking based method as follows:

	T1	T2	T3
(1)	R1(x)		
(2)			R3(y)
(3)			W3(y)
(4)	R1(y)		
(5)			W3(x)
(6)		R2(y)	
(7)			C3
(8)	W1(x)		
(9)		R2(x)	
(10)		W2(x)	
(11)	C1		
(12)		C2	

Q2.1
0.5 Points

(i) Tell if this schedule is conflict serializable or not conflict serializable and give your explanation.

- Yes, because no cycle exists in the dependency graph
- No, because there is a cycle between T1 and T3
- Yes, because it can be transformed into a serial schedule
- No, because the schedule contains conflicting operations on the same data items but no locks are used

Q2.2
0.5 Points

(ii) If in the schedule W3(x) of T3 is changed into R3(x), is this changed schedule conflict serializable or not conflict serializable? Why?

- Yes, because the change removes the cycle
- No, because the dependency still exists
- Yes, because writes are not allowed
- No, because reads increase dependencies

Q2.3
0.5 Points

If in the schedule W3(x) of T3 is changed into W3(y), is this changed schedule conflict serializable or not conflict serializable? Why?

- Yes, because the change removes the cycle
- No, because there still exists cycle in the schedule's dependency graph
- Yes, because writes are not allowed
- No, because reads increase dependencies

Q2.4
0.5 Points

In a precedence (dependency) graph, there is an edge Ti → Tj if and only if:

- Ti reads any item before Tj starts
- Ti and Tj access any common item, in any order
- An operation of Ti conflicts with one of Tj and Ti's op appears earlier in the schedule
- Ti starts before Tj starts

Q2.5
0.5 Points

A schedule's precedence graph has edges T2 → T1 and T1 → T3 (no other edges). Which serial order is equivalent to this schedule?

- T1, T2, T3
- T2, T1, T3
- T2, T3, T1
- T3, T2, T1

Q3 More about Transactions

5 Points

Q3.1

0.5 Points

A transaction is the basic unit of change and:

- May be partially applied if logging is enabled
- Must include exactly one SQL statement
- Partial transactions are not allowed
- Can include OS/file actions as part of its DB semantics

Q3.2

0.5 Points

According to the lecture, a schedule is judged correct if:

- It has no temporary inconsistencies
- It contains no conflicting operations
- It uses locks on every operation
- It is equivalent to some serial execution

Q3.3
0.5 Points

According to the lecture, which statement is correct?

- Atomicity is mainly the application's responsibility; the DBMS only logs as a convenience.
- Transactional consistency is mainly the application's responsibility (assuming the DB starts consistent and the txn runs alone).
- Isolation is mainly the application's responsibility; the DBMS doesn't automatically enforce it.
- Durability is mainly the application's responsibility; a plain write() call is already persistent.

Q3.4
0.5 Points

Reading a value written by an uncommitted transaction is the anomaly called:

- Unrepeatable read
- Dirty read
- Lost update
- Phantom read

Q3.5
0.5 Points

Two schedules are conflict equivalent if they:

- Involve the same actions of the same transactions and preserve the order of every conflicting pair
- Always produce the same final values for one specific initial state
- Preserve the order of all operations, including non-conflicting ones
- Have no lost updates

Q3.6
0.5 Points

Two operations are said to conflict when:

- They are by the same transaction and at least one is a write
- They are by different transactions on the same object and at least one is a write
- They are by different transactions on different objects
- They are both reads on the same object

Q3.7
0.5 Points

Which relationship between serializability classes is correct?

- Every serializable schedule is conflict serializable
- Conflict serializability is more general than serializability
- Every conflict-serializable schedule is not serializable
- Some schedules are serializable but not conflict serializable

Q3.8
0.5 Points

Two schedules are equivalent if:

- They have the same order of all operations
- For any database state, their effects are identical
- Each of them is equivalent to some serial schedule
- They both avoid deadlocks

Q3.9
0.5 Points

A schedule is conflict serializable if:

- It is equivalent to some serial schedule
- It does involve deadlock
- You are able to transform it into a serial schedule by swapping consecutive nonconflicting operations of different transactions.
- There is at least one cycle in the schedule’s dependency graph

Q3.10
0.5 Points

2. Assume transaction T1 performs a funds transfer between two accounts
   T1 & T2:
   Read(A)
   A = A - 500
   Write(A)
   Read(B)
   B = B + 500
   Write(B)

Which of the following transactions T2, if run concurrently with T1 without concurrency control, could interfere with T1's correctness?

Read(C); C = C + 200; Write(C)

Read(B); B = B + 100; Write(B)

Read(A); Compute interest = A * 0.05; Print(interest)

Read(D); D = D * 2; Write(D)

Q4 Crash Recovery

5 Points

Q4.1

1 Point

Which statements are correct about crash recovery?

- NO FORCE, STEAL has relatively worse runtime performance, but less work to do for crash recovery.
- NO STEAL, FORCE has relatively better runtime performance, but more work to do for crash recovery.
- Write-ahead log is a way to implement the NO FORCE, STEAL policy, so that each transaction must be logged before being committed.
- Write-ahead log is a way to implement the NO STEAL, FORCE policy, so that each transaction must be logged before being committed.

Q4.2

1 Point

Under the write-ahead logging (WAL)WAL design in the lecture, the buffer-pool policy combination used is:

- NO-STEAL + FORCE
- NO-STEAL + NO-FORCE
- STEAL + FORCE
- STEAL + NO-FORCE

Q4.3
1 Point

According to the WAL protocol, a transaction is considered committed only after the DBMS:

- Flushes all of its dirty data pages to disk
- Writes at least one log record of the transaction
- Writes a <COMMIT> record and ensures all its log records are on stable storage
- Copies updated pages back into the buffer pool

Q4.4
1 Point

In the context of Write-Ahead Logging (WAL), which statement about checkpoints is correct?

- Checkpoints eliminate the need for maintaining a write-ahead log, since all updates are flushed to disk.
- Taking frequent checkpoints always improves performance because it minimizes runtime I/O overhead.
- Checkpoints are taken only after all transactions have committed to guarantee a consistent database state.
- During a checkpoint, the DBMS flushes modified pages and log records to stable storage to limit how far back recovery must read in the log.

Q4.5
1 Point

Which statement correctly describes recovery with Write-Ahead Log in the ARIES recovery mechanism?

- We need to undo all transactions that have been committed before the crash point.
- We need to redo all transactions that didn’t commit before the crash point.
- We need to undo all transactions that didn’t commit before the crash point.
- We need to redo all transactions that were active but not committed before the crash point.

Q5 Relational Operator Implementation and Query Optimization

5 Points

Q5.1

0.5 Points

Please explain the steps of a Nested Loop Join between table1 and table2:

- Loop through each row of table1 and probe a hash table built on table2 for the join attribute. Typically, table1 is the larger table
- Loop through each row of table1 and pass that to a nested loop that loops through all rows of table2 to process the join attribute. Typically table1 is the smaller table
- Loop through each row of table1 and pass that to a nested loop that loops through all rows of table2 to process the join attribute. Typically, table1 is the larger table
- Loop through each row of table1 after sorting both tables on the join attribute, then advance pointers in order to process matches. Typically sorting replaces the need for nested loops.

Q5.2
0.5 Points

Please explain the steps of a Grace Hash Join:

Scan and build hash tables for both table1 and table2 using the join attribute and keep all partitions in memory. Then use the same hash function and take a tuple from table1 and match it to any tuple from table2 across all partitions, skipping the per-partition probe step

Scan and build sorted runs for table1 and table2 on the join attribute and output the results to the disk. Then use a merge phase to take a tuple from table1 and match it to its corresponding tuple from table2, without using any hash functions.

Scan and build a single in-memory hash table for table1 using the join attribute and keep table2 unchanged. Then use the same hash function and take a tuple from table2 and try to match it directly in table1's hash table without any partitioning to disk

Scan and build a hash table for table1 using the join attribute and output the results to the disk and do the same for table2. Then use a different hash function and take a tuple from table1 and match it to its corresponding tuple from table2 in the tables

Q5.3
0.5 Points

Please explain the steps of a Sort Merge Join:

- Sort only table1 on the join attribute. Then step through table1 and for each tuple linearly scan table2 to find the matching tuples
- Sort both table1 and table2 on the join attribute. Then step through both tables in parallel and match the matching tuples
- Sort both table1 and table2 on the join attribute. Then probe a hash table built on table2 instead of stepping through both tables in parallel.
- Sort table1 on the join attribute and keep table2 unsorted. Then step through table1 in order and match tuples from table2 by repeated binary searches

Q5.4
0.5 Points

How is the GROUP BY operator typically implemented in physical query plans?

- Using hashing or sorting
- Through Cartesian product
- With nested loops
- By dividing the table

Q5.5
0.5 Points

A file consists of 8,000 disk pages. The system has 9 buffer pages available in memory. You use the standard external merge sort algorithm with:

Initial runs generated using simple in-memory sorting
Multi-way merging with all buffer pages
Each pass reads and writes the entire file
All sorted data should be persisted to disk

How many total I/O operations (reads + writes) are required to completely sort the file?

- 96,000
- 80,000
- 64,000
- 32,000
- 16,000

Q5.6
0.5 Points

Which operator implementation is likely to perform worst on a highly skewed join attribute distribution?

- Hash Join
- Sort-Merge Join
- Index Nested Loop
- Nested Loop with No Index

Q5.7
1 Point

Which of the following is a valid equivalence in relational algebra?

$\sigma_{cond1 \wedge cond2}(R) \equiv \sigma_{cond1}(\sigma_{cond2}(R))$

$\pi_{A,B}(\sigma_{cond}(R)) \equiv \sigma_{cond}(\pi_{A,B}(R))$

$R \cup S \equiv R \cap S$

$R - S \equiv S - R$

Q5.8
1 Point

If C is the only shared attribute between R and S, then
$\pi_{S.B}\sigma_{R.A<t}(R \bowtie_{R.C=S.C} S) \equiv$

$\pi_{S.B}((\sigma_{R.A<t}(R)) \bowtie S)$

$R \bowtie \sigma_{R.A<t}(\pi_{S.B}S)$

$\sigma_{R.A<t}(R) \bowtie (\pi_{S.B}S)$

$R \times S$

Assignment 4

● Ungraded

Student
Rishith Ashit Mody

Total Points
- / 20 pts

Question 1

Transaction Management — 2.5 pts

1.1	(no title)	0.5 pts
1.2	(no title)	0.5 pts
1.3	(no title)	0.5 pts
1.4	(no title)	0.5 pts
1.5	(no title)	0.5 pts

Question 2

(no title) — 2.5 pts

2.1	(no title)	0.5 pts
2.2	(no title)	0.5 pts
2.3	(no title)	0.5 pts
2.4	(no title)	0.5 pts
2.5	(no title)	0.5 pts

Question 3

More about Transactions — 5 pts

3.1	(no title)	0.5 pts
3.2	(no title)	0.5 pts
3.3	(no title)	0.5 pts
3.4	(no title)	0.5 pts
3.5	(no title)	0.5 pts
3.6	(no title)	0.5 pts
3.7	(no title)	0.5 pts
3.8	(no title)	0.5 pts
3.9	(no title)	0.5 pts
3.10	(no title)	0.5 pts

Question 4

Crash Recovery — 5 pts

4.1	(no title)	1 pt
4.2	(no title)	1 pt
4.3	(no title)	1 pt
4.4	(no title)	1 pt
4.5	(no title)	1 pt

Question 5

Relational Operator Implementation and Query Optimization — 5 pts

5.1	(no title)	0.5 pts
5.2	(no title)	0.5 pts
5.3	(no title)	0.5 pts
5.4	(no title)	0.5 pts
5.5	(no title)	0.5 pts
5.6	(no title)	0.5 pts
5.7	(no title)	1 pt
5.8	(no title)	1 pt